

Guide d'exploitation - DataVision AI v2.81.0

1. Objectif

Ce guide définit les opérations récurrentes nécessaires pour maintenir DataVision disponible, sûr, sauvegardé et auditable.

2. Contrôles quotidiens

Vérifier :

- /health/ready ;
- erreurs API/worker ;
- profondeur et ancienneté de la queue Redis ;
- jobs en échec/retry ;
- état PostgreSQL ;
- disponibilité du sandbox ;
- fraîcheur des backups ;
- SLO et alertes ;
- KMS/Vault ;
- antivirus lorsque requis ;
- expiration certificats/TLS via l'outillage d'infrastructure.

3. Gestion des comptes

- créer un compte nominatif par utilisateur ;
- appliquer le moindre privilège ;
- exiger MFA pour les comptes sensibles ;
- revoir périodiquement les membres des workspaces ;
- désactiver immédiatement un compte sortant ;
- révoquer ses sessions actives.

4. Sauvegarde

Conserver plusieurs générations et au moins une copie hors du domaine de panne principal. Une sauvegarde n'est considérée fiable qu'après test de restauration.

Journaliser : date, version DataVision, schéma, taille, digest, emplacement et résultat du restore drill.

5. Restauration

1. isoler l'instance cible ;
2. vérifier compatibilité version/schema ;
3. restaurer ;
4. exécuter migrations nécessaires ;

5. vérifier readiness ;
6. exécuter smoke tests ;
7. rouvrir progressivement le trafic ;
8. enregistrer l'opération dans l'audit.

6. Rotation des secrets

Planifier la rotation de :

- AUTH_SECRET ;
- secrets connecteurs ;
- tokens OIDC/SCIM ;
- webhooks ;
- clés KMS/Vault ;
- secrets métriques et bootstrap selon la politique.

Après rotation, tester login, refresh, connecteurs, jobs, KMS et readiness.

7. Incident sécurité

1. qualifier la sévérité ;
2. isoler le service/workspace concerné ;
3. révoquer sessions/tokens ;
4. préserver logs et preuves ;
5. tourner les secrets exposés ;
6. déterminer données/comptes impactés ;
7. appliquer correction ;
8. restaurer progressivement ;
9. produire RCA et actions préventives.

8. Vulnérabilité de dépendance

Lorsqu'un avis High/Critical apparaît :

1. identifier package et chemins d'utilisation ;
2. confirmer exploitabilité ;
3. mettre à jour le pin ;
4. lancer pip check/typecheck/tests ;
5. reconstruire les images ;
6. rescanner ;
7. publier un nouveau digest ;
8. documenter la décision.

9. Upgrade

Toujours utiliser un nouveau dossier de code. Ne pas écraser une ancienne release. Sauvegarder avant migration, valider le nouvel environnement, puis seulement déplacer le trafic.

10. Commandes d'exploitation

```
python scripts\repository_hygiene.py --root . --check  
python scripts\dependency_compatibility.py --root . --check  
python scripts\security_hardening_audit.py --root . --check  
python scripts\production_baseline.py --root . --check  
python scripts\config_doctor.py --root . --check  
docker compose config -q  
docker compose up -d --build
```

11. Contrôle avant ouverture du trafic

- auth testée ;
- workspace/roles testés ;
- backup disponible ;
- migrations prêtes ;
- readiness verte ;
- connecteurs critiques testés ;
- logs/alertes visibles ;
- secret bootstrap protégé ;
- aucun secret d'exemple ;
- scans de sécurité traités.